

Ethan Patterson

HW0- Problem 2

a.) An algorithm that checks finds the two smallest numbers in a series of inputs. It iterates over the list of numbers, checks if the current number is smaller than the current smallest, and if it is, replaces it with the current and changes second smallest to the previous "smallest". If it isn't, it checks if current is less than second smallest (and not equal to the smallest, no duplicates allowed). In this case, the second smallest gets changed to current and the previous second smallest gets banished to the void. After iterating over all the inputs, the smallest and second smallest are returned.

b.)

```
Int[] inputarray
```

```
Smallest = integer_max
```

```
Second_smallest = integer_max
```

```
For value in inputarray
```

```
    If value less than smallest
```

```
        Second_smallest = smallest
```

```
        Smallest = value
```

```
    Elif value less than second_smallest and value isnt equal to smallest
```

```
        Second smallest = value
```

```
Print smallest
```

```
Print second smallest
```

c.) Every value gets compared to the smallest and second smallest. If it's less than either of those (with smallest taking priority), the values get updated accordingly.

One note here is that this works up until integer limit, if all the values are above the integer limit, this breaks. Switching to a long would fix that, but you're good up until that limit. There's no good solution as there'll always be a max limit, so for now just keep inputs below 2,147,483,647 and we won't be having problems here.

d.) $O(n)$

e.) We iterate over the entire list once. Calculations are done during the iterating, and those can be simplified to just be a C value multiplier for our o notation, and thus not relevant. As such, we're simply just n .